

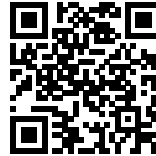
Module 6: Gradient Descent

Instructional Hours: 4

1 Introduction to optimization

Video Visit the URL below to view a video:

<https://www.youtube.com/embed/n-Y0SDS0fUI>



Introduction to optimization

1.1 Content Curated Video

Video Visit the URL below to view a video:

<https://www.youtube.com/embed/qg4PchTECck>



Illustration of Gradient Descent

2 Fundamental Concepts and Terminology in Optimization

Optimization involves finding the best solution from a set of feasible solutions. It is crucial in fields such as engineering, economics, and machine learning. It helps to understand how changes in the parameters affect the outcome.

3 Multivariable Optimization

Multivariable optimization involves finding the maximum or minimum of a function that depends on two or more variables. Unlike single-variable optimization, which deals with functions of one variable, multivariable optimization requires analyzing how changes in multiple input variables affect the output.

4 Objective Function

The function that is being optimized is called the objective function. In the context of multivariable optimization, this function is typically denoted as $f(x_1, x_2, \dots, x_n)$, where $x_1, x_2, \dots, x_n$ are the variables.

5 Gradient

The gradient of a function is a vector of partial derivatives. For a function $f(x_1, x_2, \dots, x_n)$, the gradient ∇f is given by:

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right).$$

The gradient points in the direction of the steepest ascent of the function. Its magnitude indicates how steep the ascent is.

6 Optimization Problems

- Unconstrained Optimization: Finding the extrema (maxima or minima) of the objective function without any restrictions on the variables.
- Constrained Optimization: Finding the extrema of the objective function subject to one or more constraints. Constraints are usually given as equations or inequalities that the solution must satisfy.

7 Gradient Descent Algorithm

An iterative method used to find local minima of a function. Starting from an initial guess, the algorithm updates the variables by moving in the direction opposite to the gradient:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \eta \nabla f(\mathbf{x}_k),$$

where η is the learning rate, a small positive number that controls the step size.

8 Learning Rate

A parameter in the gradient descent algorithm that determines the size of the steps taken towards the minimum. Choosing an appropriate learning rate is crucial; too large a rate might overshoot the minimum, while too small a rate might slow down convergence.

9 Convergence Criteria

The conditions under which the gradient descent algorithm is considered to have converged. Common criteria include a sufficiently small change in the objective function value or a sufficiently small norm of the gradient.

10 Constraints Handling

Techniques used to ensure that the solutions satisfy any constraints imposed by the problem. This can involve methods like Lagrange multipliers for equality constraints or penalty functions for inequality constraints.

Understanding these concepts provides a foundation for effectively applying optimization techniques to various practical problems involving optimizing functions in machine learning.

11 Significance and Problem Types

- Design Optimization: It entails adjusting parameters to achieve the best performance.
- Economic Planning: This involves maximizing profit or minimizing costs based on multiple influencing factors.
- Machine Learning: This involves tuning models with multiple hyperparameters to improve performance.

12 Optimization of functions of one variable

In a one-dimensional context, we focus on minimizing a function $f(x)$ where x is a real number.

Example 1. Minimize $f(x) = x^2 + 4x + 4$.

Solution

1. *Objective Function:*

$$f(x) = x^2 + 4x + 4$$

2. *Find the minimum by completing the square:*

$$f(x) = (x + 2)^2$$

3. *The minimum value is at:*

$$x = -2$$

4. *Function value at minimum:*

$$f(-2) = 0$$

Example 2. Minimize $f(x) = x^2 + 1$ with the constraint $x \geq 0$.

Solution:

1. *Unconstrained minimum:*

$$x = 0$$

2. *Function value at $x = 0$:*

$$f(0) = 1$$

3. *Since $x \geq 0$ is satisfied, the minimum is:*

$$x = 0, \quad f(0) = 1$$

13 Gradient Descent Algorithm

Gradient descent is an iterative optimization algorithm used to find the local minimum of a function. The method involves updating the current estimate in the direction of the negative gradient.

13.1 Step-by-Step Gradient Descent

Gradient Descent Formula:

$$x_{k+1} = x_k - \eta \cdot \frac{df}{dx}$$

where η is the learning rate.

Example 3. Minimize $f(x) = x^2 + 2x + 1$ starting from $x_0 = 0$ with $\eta = 0.1$.

Solution

1. *Objective Function:*

$$f(x) = x^2 + 2x + 1$$

2. *Gradient:*

$$\frac{df}{dx} = 2x + 2$$

3. *Update Rule:*

$$x_{k+1} = x_k - 0.1 \cdot (2x_k + 2)$$

4. *First Iteration:*

$$x_1 = 0 - 0.1 \cdot (2 \cdot 0 + 2) = 0 - 0.2 = -0.2$$

5. *Second Iteration:*

$$\begin{aligned} \frac{df}{dx} \text{ at } x_1 &= 2(-0.2) + 2 = 1.6 \\ x_2 &= -0.2 - 0.1 \cdot 1.6 = -0.2 - 0.16 = -0.36 \end{aligned}$$

Example 4. Minimize $f(x) = \sin(x)$ starting from $x_0 = \pi/2$ with $\eta = 0.01$.

Solution

1. *Objective Function:*

$$f(x) = \sin(x)$$

2. *Gradient:*

$$\frac{df}{dx} = \cos(x)$$

3. *Update Rule:*

$$x_{k+1} = x_k - 0.01 \cdot \cos(x_k)$$

4. *First Iteration:*

$$x_1 = \frac{\pi}{2} - 0.01 \cdot \cos\left(\frac{\pi}{2}\right) = \frac{\pi}{2}$$

5. *Observation:* As $\cos\left(\frac{\pi}{2}\right) = 0$, there is no change. Consider starting closer to points where $\cos(x)$ is significant.

14 Implementing Gradient Descent

14.1 Step Size (Learning Rate) Selection

The choice of learning rate η affects convergence speed and stability. Too large η may lead to divergence, while too small η may result in slow convergence.

Example 5. Minimize $f(x) = (x - 1)^2$ with learning rates $\eta = 0.1$ and $\eta = 0.5$, starting from $x_0 = 0$.

Solution: Solution with $\eta = 0.1$:

1. Gradient:

$$\frac{df}{dx} = 2(x - 1)$$

2. Update Rule:

$$x_{k+1} = x_k - 0.1 \cdot 2(x_k - 1)$$

3. First Iteration:

$$x_1 = 0 - 0.1 \cdot 2(0 - 1) = 0 + 0.2 = 0.2$$

Solution with $\eta = 0.5$:

1. Update Rule:

$$x_{k+1} = x_k - 0.5 \cdot 2(x_k - 1)$$

2. First Iteration:

$$x_1 = 0 - 0.5 \cdot 2(0 - 1) = 0 + 1 = 1$$

Example 6. Check convergence of gradient descent for $f(x) = x^2 - 4x + 4$ with $\eta = 0.1$ and initial point $x_0 = 0$.

Solution

1. Objective Function:

$$f(x) = x^2 - 4x + 4$$

2. Gradient:

$$\frac{df}{dx} = 2x - 4$$

3. *Update Rule:*

$$x_{k+1} = x_k - 0.1 \cdot (2x_k - 4)$$

4. *First Iteration:*

$$x_1 = 0 - 0.1 \cdot (2 \cdot 0 - 4) = 0 + 0.4 = 0.4$$

5. *Second Iteration:*

$$x_2 = 0.4 - 0.1 \cdot (2 \cdot 0.4 - 4) = 0.4 - 0.1 \cdot (-3.2) = 0.4 + 0.32 = 0.72$$

15 Application

15.1 Local Minima

Gradient descent can get stuck in local minima. To handle this, try different starting points or use methods like simulated annealing.

Example 7. Minimize $f(x) = x^4 - 3x^3 + 2$ with $\eta = 0.01$, starting from $x_0 = 0$.

Solution:

1. *Objective Function:*

$$f(x) = x^4 - 3x^3 + 2$$

2. *Gradient:*

$$\frac{df}{dx} = 4x^3 - 9x^2$$

3. *Update Rule:*

$$x_{k+1} = x_k - 0.01 \cdot (4x_k^3 - 9x_k^2)$$

4. *First Iteration:*

$$x_1 = 0 - 0.01 \cdot (4 \cdot 0^3 - 9 \cdot 0^2) = 0$$

Observation: The gradient is zero at x_0 . Consider different initial points to find other minima.

Example 8. *Problem: Minimize $f(x) = x^2 - 4x + 5$ with different initial guesses $x_0 = 0$ and $x_0 = 10$, using $\eta = 0.1$.*

Solution: *Solution with $x_0 = 0$:*

1. *Gradient:*

$$\frac{df}{dx} = 2x - 4$$

2. *Update Rule:*

$$x_{k+1} = x_k - 0.1 \cdot (2x_k - 4)$$

3. *First Iteration:*

$$x_1 = 0 - 0.1 \cdot (2 \cdot 0 - 4) = 0 + 0.4 = 0.4$$

Solution with $x_0 = 10$:

1. *Update Rule:*

$$x_{k+1} = x_k - 0.1 \cdot (2x_k - 4)$$

2. *First Iteration:*

$$x_1 = 10 - 0.1 \cdot (2 \cdot 10 - 4) = 10 - 0.1 \cdot 16 = 10 - 1.6 = 8.4$$

Observation: *Different initial guesses lead to the same minimum but with different paths.*

Video Visit the URL below to view a video:

<https://www.youtube.com/embed/0kFydRfswU8>



Solving an equation using Gradient Descent

15.2 Optimization of functions of two variables

In a two-dimensional context, we focus on optimizing functions of the form $f(x, y)$, where x and y are the variables. The goal is to find the point (x, y) that minimizes the function $f(x, y)$.

15.2.1 Gradient and Gradient Descent

The gradient of a function $f(x, y)$ is a vector that points in the direction of the steepest ascent. For optimization, we use the negative gradient to update our estimates in the direction of the steepest descent.

$$\nabla f(x, y) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$$

The gradient descent update rule is:

$$(x_{k+1}, y_{k+1}) = (x_k, y_k) - \eta \nabla f(x_k, y_k)$$

where η is the learning rate.

15.2.2 Gradient Descent Algorithm

Given a function $f(x, y)$, the gradient descent algorithm updates the variables iteratively:

1. Compute the gradient $\nabla f(x, y)$.
2. Update the variables using:

$$\begin{aligned} x_{k+1} &= x_k - \eta \frac{\partial f}{\partial x} \\ y_{k+1} &= y_k - \eta \frac{\partial f}{\partial y} \end{aligned}$$

3. Repeat until convergence.

Example 9. Problem: Find the gradient of $f(x, y) = e^x + \sin(y)$.

Solution:

1. Compute the partial derivatives:

$$\begin{aligned} \frac{\partial f}{\partial x} &= e^x \\ \frac{\partial f}{\partial y} &= \cos(y) \end{aligned}$$

2. The gradient is:

$$\nabla f(x, y) = (e^x, \cos(y))$$

Example 10. Minimize $f(x, y) = x^2 + y^2 - 4x - 6y + 13$ with an initial guess $(x_0, y_0) = (0, 0)$ and learning rate $\eta = 0.1$.

Solution:

1. *Objective Function:*

$$f(x, y) = x^2 + y^2 - 4x - 6y + 13$$

2. *Gradient:*

$$\begin{aligned}\frac{\partial f}{\partial x} &= 2x - 4 \\ \frac{\partial f}{\partial y} &= 2y - 6\end{aligned}$$

3. *Update Rule:*

$$\begin{aligned}x_{k+1} &= x_k - 0.1 \cdot (2x_k - 4) \\ y_{k+1} &= y_k - 0.1 \cdot (2y_k - 6)\end{aligned}$$

4. *First Iteration:*

$$\begin{aligned}x_1 &= 0 - 0.1 \cdot (2 \cdot 0 - 4) = 0 + 0.4 = 0.4 \\ y_1 &= 0 - 0.1 \cdot (2 \cdot 0 - 6) = 0 + 0.6 = 0.6\end{aligned}$$

Example 11. Minimize $f(x, y) = e^x + x^2y + y^2$ with an initial guess $(x_0, y_0) = (1, 1)$ and learning rate $\eta = 0.01$.

Solution:

1. *Objective Function:*

$$f(x, y) = e^x + x^2y + y^2$$

2. *Gradient:*

$$\begin{aligned}\frac{\partial f}{\partial x} &= e^x + 2xy \\ \frac{\partial f}{\partial y} &= x^2 + 2y\end{aligned}$$

3. *Update Rule:*

$$\begin{aligned}x_{k+1} &= x_k - 0.01 \cdot (e^{x_k} + 2x_ky_k) \\ y_{k+1} &= y_k - 0.01 \cdot (x_k^2 + 2y_k)\end{aligned}$$

4. *First Iteration:*

$$x_1 = 1 - 0.01 \cdot (e^1 + 2 \cdot 1 \cdot 1) = 1 - 0.01 \cdot (e + 2) \approx 1 - 0.01 \cdot 4.718 = 1 - 0.047 = 0.953$$

$$y_1 = 1 - 0.01 \cdot (1^2 + 2 \cdot 1) = 1 - 0.01 \cdot 3 = 1 - 0.03 = 0.97$$

15.3 Optimization of functions of three variables

In a three-dimensional context, we focus on optimizing functions of the form $f(x, y, z)$, where x, y , and z are the variables. The goal is to find the point (x, y, z) that minimizes the function $f(x, y, z)$.

15.3.1 Gradient Descent Algorithm

1. Initialization

- Start with an initial guess for the variables $x^{(0)}, y^{(0)}, z^{(0)}$.

2. Compute the Gradient

- Calculate the gradient of the function $f(x, y, z)$ at the current point. The gradient ∇f is a vector of partial derivatives:

$$\nabla f(x, y, z) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z} \right)$$

3. Update the Variables

- Update the variables by moving in the direction opposite to the gradient:

$$\begin{aligned} \mathbf{x}^{(k+1)} &= \mathbf{x}^{(k)} - \eta \frac{\partial f}{\partial \mathbf{x}} \\ y^{(k+1)} &= y^{(k)} - \eta \frac{\partial f}{\partial y} \\ z^{(k+1)} &= z^{(k)} - \eta \frac{\partial f}{\partial z} \end{aligned}$$

where η is the learning rate.

4. Check Convergence

- Repeat steps 2 and 3 until convergence criteria are met (e.g., when changes in function values or variables are below a threshold).

Example 12. Minimize the function $f(x, y, z) = (x - 1)^2 + (y - 2)^2 + (z - 3)^2$.
Solution

1. Gradient Computation:

$$\frac{\partial f}{\partial x} = 2(x - 1), \quad \frac{\partial f}{\partial y} = 2(y - 2), \quad \frac{\partial f}{\partial z} = 2(z - 3)$$

2. Update Rule: Choose $\eta = 0.1$. Start with $(x, y, z) = (0, 0, 0)$.

Iteration 1:

$$x^{(1)} = 0 - 0.1 \cdot 2 \cdot (-1) = 0.2$$

$$y^{(1)} = 0 - 0.1 \cdot 2 \cdot (-2) = 0.4$$

$$z^{(1)} = 0 - 0.1 \cdot 2 \cdot (-3) = 0.6$$

Repeat until (x, y, z) converges to (1, 2, 3).

16 Types of Gradient Descent Approaches

There are several types of gradient descent approaches, each with its characteristics and use cases.

16.1 Batch Gradient Descent

Batch Gradient Descent computes the gradient of the cost function with respect to the parameters for the entire training dataset. It then updates the parameters by moving in the direction opposite to the gradient.

16.1.1 Update Rule:

$$\theta := \theta - \eta \nabla J(\theta)$$

where $\nabla J(\theta)$ is the gradient of the cost function $J(\theta)$ with respect to the parameters θ , and η is the learning rate.

16.1.2 Characteristics:

- Requires the entire dataset to be processed before making a single update.
- Can be computationally expensive for large datasets.
- Provides stable convergence but may be slow due to frequent processing of the entire dataset.

17 Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent updates the parameters using the gradient computed from a single data point (or a few data points) at a time. This approach introduces more noise but can lead to faster convergence.

17.0.1 Update Rule:

$$\theta := \theta - \eta \nabla J_i(\theta)$$

where $\nabla J_i(\theta)$ is the gradient of the cost function with respect to the parameters θ computed from the i -th data point.

17.0.2 Characteristics:

- Updates are made more frequently, which can lead to faster convergence.
- Introduces noise into the gradient estimates, which can help in escaping local minima but may lead to oscillations.
- More suitable for online learning scenarios where data arrives in a stream.

18 Mini-Batch Gradient Descent

Mini-Batch Gradient Descent is a compromise between Batch Gradient Descent and Stochastic Gradient Descent. It updates the parameters using a small, random subset (mini-batch) of the data.

18.0.1 Update Rule:

$$\theta := \theta - \eta \nabla J_{\text{mini-batch}}(\theta)$$

where $\nabla J_{\text{mini-batch}}(\theta)$ is the gradient computed from the mini-batch of data points.

18.0.2 Characteristics:

- Balances the efficiency of Batch Gradient Descent with the speed of Stochastic Gradient Descent.
- Reduces the noise introduced by SGD while still benefiting from frequent updates.
- Commonly used in practice due to its efficiency and convergence properties.

19 Momentum-Based Gradient Descent

Momentum-Based Gradient Descent accelerates gradient descent by adding a fraction of the previous update to the current update. This helps in smoothing out the updates and can accelerate convergence.

19.0.1 Update Rule:

$$\begin{aligned} \mathbf{v}^{(k)} &= \beta \mathbf{v}^{(k-1)} + (1 - \beta) \nabla J(\theta) \\ \theta^{(k+1)} &= \theta^{(k)} - \eta \mathbf{v}^{(k)} \end{aligned}$$

where $\mathbf{v}^{(k)}$ is the velocity (momentum), β is the momentum term, and η is the learning rate.

19.0.2 Characteristics:

- Helps accelerate convergence in the relevant direction and dampens oscillations.
- Requires tuning of the momentum parameter β .

20 Nesterov Accelerated Gradient (NAG)

Nesterov Accelerated Gradient is an improvement over traditional momentum methods. It includes a look-ahead step that provides a more accurate estimate of the gradient.

20.0.1 Update Rule:

$$\begin{aligned} \mathbf{v}^{(k)} &= \beta \mathbf{v}^{(k-1)} + \eta \nabla J(\boldsymbol{\theta} - \beta \mathbf{v}^{(k-1)}) \\ \boldsymbol{\theta}^{(k+1)} &= \boldsymbol{\theta}^{(k)} - \mathbf{v}^{(k)} \end{aligned}$$

20.0.2 Characteristics:

- Provides faster convergence compared to standard momentum.
- Requires careful tuning of β and η .

21 Adagrad (Adaptive Gradient Algorithm)

Adagrad adapts the learning rate for each parameter based on the historical gradients. Parameters with frequent updates receive smaller learning rates, while parameters with infrequent updates receive larger learning rates.

21.0.1 Update Rule:

$$\theta := \theta - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot \nabla J(\theta)$$

where G_t is the sum of squares of past gradients, ϵ is a small constant to prevent division by zero, and $\odot$ denotes element-wise multiplication.

21.0.2 Characteristics:

- Automatically adjusts learning rates based on parameter updates.
- Can be beneficial for sparse data but may have issues with very small learning rates over time.

22 RMSprop (Root Mean Square Propagation)

RMSprop modifies Adagrad by using a moving average of squared gradients to adjust the learning rate. This prevents the learning rate from shrinking too quickly.

22.0.1 Update Rule:

$$\begin{aligned} E[g^2]_t &= \rho E[g^2]_{t-1} + (1 - \rho)(\nabla J(\theta))^2 \\ \theta &= \theta - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} \odot \nabla J(\theta) \end{aligned}$$

where $E[g^2]_t$ is the moving average of squared gradients, ρ is the decay factor, and ϵ is a small constant.

22.0.2 Characteristics:

- Addresses the issue of decreasing learning rates in Adagrad.
- Often leads to faster convergence in practice.

23 Adam (Adaptive Moment Estimation)

Adam combines the advantages of two other extensions: Adaptive Gradient Algorithm (Adagrad) and Momentum. It computes adaptive learning rates for each parameter by combining the benefits of both techniques.

23.0.1 Update Rule:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) \nabla J(\theta) \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) (\nabla J(\theta))^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t} \\ \hat{v}_t &= \frac{v_t}{1 - \beta_2^t} \\ \theta &= \theta - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \end{aligned}$$

where m_t is the first moment (mean), v_t is the second moment (uncentered variance), $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected estimates, and ϵ is a small constant.

23.0.2 Characteristics:

- Combines the benefits of both RMSprop and momentum.
- Adaptive learning rates and momentum lead to fast convergence.
- Widely used in practice due to its efficiency and robustness.